Programación 1 — Unidad 6: Funciones

Actividad 4: Módulos y paquetes, librerías propias y de terceros

Índice

– 6.9. Módulos: dividir el código en archivos

– 6.10. Paquetes: organizar módulos en carpetas

– 6.11. Entornos virtuales e instalación de librerías

6.9. Módulos: dividir el código en archivos

Un **módulo** es, simplemente, un archivo `.py` con código (funciones, variables) que podemos usar desde otro archivo. Dividir un programa en módulos lo hace más ordenado: cada archivo se ocupa de un tema. Para usar el contenido de un módulo, se lo **importa** con `import`. Supongamos un archivo `matematica.py` con funciones:

```
# archivo: matematica.py
def sumar(a, b):
    return a + b

def es_par(n):
    return n % 2 == 0
```

Qué hace el código:

- Este es un archivo aparte, llamado `matematica.py`, que agrupa funciones relacionadas con la matemática.
- `def sumar(a, b):` y `def es_par(n):` son funciones normales, pero viven en este módulo separado.
- Por sí solo, este archivo no «hace» nada al ejecutarse: solo ofrece funciones para que otros lo usen.

Desde otro archivo, en la misma carpeta, lo importamos y usamos sus funciones:

```
# archivo: main.py
import matematica

print(matematica.sumar(2, 3))     # 5
print(matematica.es_par(10))      # True

from matematica import sumar
print(sumar(4, 4))                # 8
```

Qué hace el código:

- `import matematica` trae todo el módulo. Para usar sus funciones, se escribe `matematica.` antes del nombre.
- `matematica.sumar(2, 3)` llama a la función `sumar` que vive en ese módulo.
- `from matematica import sumar` trae **solo** la función `sumar`, sin tener que escribir el prefijo.
- Por eso después podemos llamarla directamente como `sumar(4, 4)`.

Nota

Python trae una enorme **biblioteca estándar** de módulos ya listos: `math` (matemática), `random` (azar), `datetime` (fechas) y muchos más. Se importan igual: `import random` y después `random.randint(1, 6)` para simular el tiro de un dado.

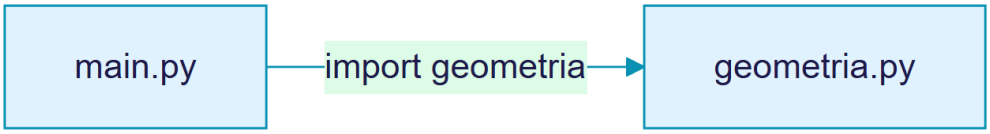
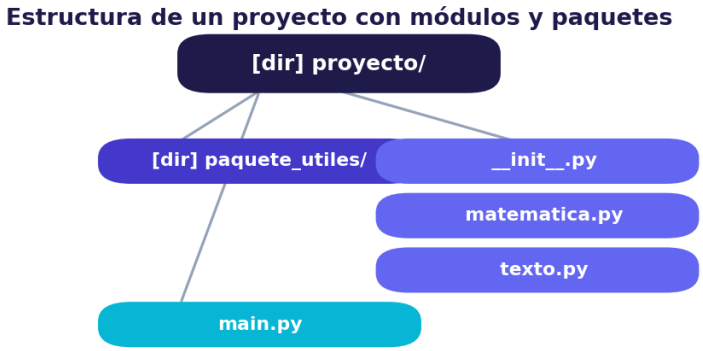


Diagrama 6.9 generado con Mermaid.

6.10. Paquetes: organizar módulos en carpetas

Cuando hay muchos módulos, se agrupan en **paquetes**: carpetas que contienen módulos relacionados. Tradicionalmente, una carpeta se reconoce como paquete si incluye un archivo especial llamado `__init__.py` (que puede estar vacío). Así se arma una jerarquía ordenada.



Un proyecto organizado en un paquete con varios módulos y un archivo principal que los usa.

```
proyecto/
├── main.py
└── paquete_utiles/
```

```
    ├── __init__.py
    ├── matematica.py
    └── texto.py
```

Qué hace el código:

- Esto no es código, sino la **estructura de carpetas** de un proyecto ordenado.
- `proyecto/` es la carpeta principal; adentro está `main.py`, el programa que se ejecuta.
- `paquete_utiles/` es un paquete: una carpeta con módulos relacionados (`matematica.py`, `texto.py`).
- El archivo `__init__.py` es el que le indica a Python que esa carpeta es un paquete.

Buenas prácticas

Por esta razón, a partir de esta unidad conviene que cada ejercicio viva en su propia carpeta, con un `main.py` y módulos separados (como `funciones.py`). Separar la lógica del programa principal es una costumbre profesional que vas a usar toda tu carrera.

6.11. Entornos virtuales e instalación de librerías

Además de la biblioteca estándar, existen miles de **librerías de terceros** que se instalan con una herramienta llamada `pip`. El problema es que distintos proyectos pueden necesitar versiones distintas de la misma librería, y se pisarían entre sí. La solución son los **entornos virtuales**: instalaciones de Python aisladas, una por proyecto.



Cada proyecto tiene su entorno virtual; así las librerías de uno no interfieren con las de otro.

Analogía

Un entorno virtual es como tener una caja de herramientas separada para cada proyecto. En vez de un único cajón gigante donde todo se mezcla y una herramienta vieja arruina un trabajo nuevo, cada proyecto lleva su propia caja con exactamente las versiones que necesita.

```
python -m venv venv

# Windows:
venv\Scripts\activate
# Linux o macOS:
source venv/bin/activate
```

```
pip install requests
deactivate
```

Qué hace el código:

- `python -m venv venv` crea el entorno virtual: genera una carpeta (llamada `venv`) con una copia aislada de Python.
- Las líneas de `activate` lo **activan** (son distintas según el sistema operativo). A partir de ahí, trabajás «dentro» del entorno.
- `pip install requests` instala una librería de terceros, pero **solo** dentro de este entorno, sin afectar al resto del sistema.
- `deactivate` sale del entorno virtual y vuelve a la situación normal.

Buenas prácticas

Guardá las dependencias de tu proyecto en un archivo `requirements.txt` con el comando `pip freeze > requirements.txt`. Así cualquier persona puede recrear tu entorno con `pip install -r requirements.txt`. Y nunca subas la carpeta `venv` a Git: agregala al `.gitignore`.

Encontrá el error

Este programa no explota, pero imprime «El promedio es: None». Fijate bien qué le falta a la función antes de seguir.

```
def calcular_promedio(a, b, c):
    promedio = (a + b + c) / 3

resultado = calcular_promedio(8, 7, 9)
print("El promedio es:", resultado)
```

Qué estaba mal

A la función le falta el return promedio. Sin él, la función calcula el valor pero nunca lo devuelve, así que resultado queda en None (el valor por defecto cuando una función no retorna nada).

Practicá la lectura

Explicá qué es un parámetro con valor por defecto y qué imprime cada una de las dos llamadas.

```
def potencia(base, exponente=2):
    return base ** exponente

print(potencia(5))
print(potencia(2, 10))
```

Para reflexionar y autoevaluarte

— ¿Qué ventaja concreta notaste al dividir un programa en funciones en vez de escribir todo seguido?

— ¿Te confundiste alguna vez entre imprimir un resultado dentro de una función y devolverlo con return?

Antes de seguir, verificá que podés:

— Definir una función con parámetros y un valor de retorno.

— Explicar la diferencia entre una variable local y una global (alcance).

— Documentar una función con un docstring.

— Organizar funciones relacionadas en un módulo propio.